GWORLDSOFT SOLUTIONS LIMITED

3MTT NIGERIA PARTNER PROGRAM

CAPSTONE PROJECT GUIDE · JULY 2026 COHORT

End-to-End Machine Learning: From Data Collection to Deployment

A complete, step-by-step blueprint for building and shipping a real-world ML product — for students on the Data Science / Machine Learning / AI track, and a parallel Power BI dashboard track for Data Analyst students.

TRACK A — DS / ML / AI

Kaggle data → EDA → 3 trained models → FastAPI/Flask API → Streamlit/JS UI → Cloud deployment

TRACK B — DATA ANALYST

Business dataset → Power Query → Data model → DAX → Power BI dashboard → Published URL

Godspower Uyanga

Lead AI & ML, GworldSoft Solutions Limited · Lead Data Scientist & AI Engineer, Interswitch Group · Mentor, 3MTT Nigeria

MSc Software Engineering · MSc Financial Engineering · MSc Artificial Intelligence Engineering

linkedin.com/in/godspower-uyanga-76aab01b0

July 2026

Instructor & Mentor Edition

Table of Contents

NOTE
This guide is written for the July 2026 3MTT Nigeria cohort in partnership with GworldSoft Solutions Limited. Students must submit both a working project URL/repo AND a short demo video/screenshot log, as described in Section 5.

1. Introduction & Program Objectives

This capstone is designed to take you from raw data to a live, usable product — the same journey a professional Data Scientist, ML Engineer, or AI Engineer follows in industry. By the end of this project, you will have a portfolio piece you can show to recruiters, not just a notebook.

Why this project matters

- Employers hire people who can ship, not just people who can train a model in a notebook.
- This project proves you understand the full lifecycle: data → model → API → UI → cloud.
- It gives you a real, shareable URL to put on your CV, LinkedIn, and portfolio.

Two tracks, one standard of excellence

Students choose one of two tracks based on their specialization inside 3MTT:

Track	Who it's for	Final Deliverable
TRACK A	Data Science, Machine Learning, Artificial Intelligence students	A deployed ML web app with a live URL (FastAPI/Flask backend + Streamlit/JS frontend)
TRACK B	Data Analyst students	A published, interactive Power BI dashboard with a shareable URL

What "Qualified" means at the end of this project

Completing this capstone to the standard described in this guide qualifies you as industry-ready — able to collect and clean real data, benchmark multiple models honestly, build a backend service, connect it to a usable interface, and deploy it so anyone in the world can access it. This is the bar for a junior Data Scientist / ML / AI Engineer role in 2026.

> **MENTOR TIP**
> Treat this like a real client project. Write your README as if a hiring manager is the only person who will ever read it — because they will.

2. Choosing Your Track

Use the table below to confirm you're building the right deliverable for your specialization.

Question	If yes → Track A (ML/DS/AI)	If yes → Track B (Data Analyst)
Do you write Python model training code?	✔	
Do you build dashboards and business reports?		✔
Is your 3MTT track "Data Science", "AI Engineering" or "Machine Learning"?	✔	
Is your 3MTT track "Data Analytics"?		✔
Final artifact is a hosted web app / API?	✔	
Final artifact is an interactive BI dashboard?		✔

> **NOT SURE?**
> Speak with your mentor, Godspower Uyanga, before Week 2. It is far easier to pick the right track early than to switch halfway through.

3. Track A — ML / Data Science / AI: Step-by-Step

Follow these steps in order. Each step builds on the one before it — do not skip to modeling before your data is clean.

Step 1 — Define the Problem

Before touching any data, write one paragraph answering:

- What decision or prediction will this model make? (e.g. "predict if a loan will default")
- Is this classification, regression, clustering, or a recommendation problem?
- Who is the end user of your final app?

Step 2 — Data Collection

Source a real dataset — do not fabricate data. Recommended sources:

Source	Best for	Link
Kaggle Datasets	General ML practice datasets, competitions	www.kaggle.com/datasets
UCI ML Repository	Classic, clean academic datasets	archive.ics.uci.edu
Nigeria Data Portal / NBS	Local, relevant Nigerian datasets	nigeria.opendataforafrica.org
data.gov / World Bank Open Data	Economic, health, government data	data.worldbank.org
Hugging Face Datasets	NLP and text-based datasets	huggingface.co/datasets

> **MENTOR TIP**
> Pick a dataset with at least 500–1000+ rows and a clear target column. Avoid datasets that are already "too clean" — real messiness is where you learn the most.

Step 3 — Load & Inspect the Data

```
import pandas as pd
df = pd.read_csv("data/raw_dataset.csv")
print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())
```

Step 4 — Data Cleaning

- Handle missing values: drop, mean/median/mode impute, or model-based impute.
- Remove duplicate rows.
- Fix inconsistent categories (e.g. "Male", "male", "M" → one label).
- Detect and treat outliers (IQR method or z-score).
- Convert data types correctly (dates, categories, numerics).

```
df = df.drop_duplicates()
df['income'] = df['income'].fillna(df['income'].median())
df['gender'] = df['gender'].str.lower().str.strip()
Q1, Q3 = df['income'].quantile([0.25, 0.75])
IQR = Q3 - Q1
df = df[(df['income'] >= Q1 - 1.5*IQR) & (df['income'] <= Q3 + 1.5*IQR)]
```

Step 5 — Exploratory Data Analysis (EDA)

Use visualizations to understand relationships before modeling:

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.countplot(x='target', data=df)
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
sns.boxplot(x='target', y='income', data=df)
plt.show()
```

> **DELIVERABLE**
> Save at least 4–6 EDA charts and one paragraph of insight per chart in your notebook — this becomes part of your final report.

Step 6 — Feature Engineering

- Encode categorical variables: `pd.get_dummies()` or `LabelEncoder`.
- Scale numeric features: `StandardScaler` or `MinMaxScaler`.
- Create new features where useful (e.g. "age group" from "age").
- Split data: `train_test_split(X, y, test_size=0.2, random_state=42)`.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = df.drop("target", axis=1)
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Step 7 — Train 3 Models & Choose the Best One

Every student must train at least 3 different model families, compare them fairly on the same test set, and justify the final choice — this is the single most important step for your grade.

Model	Good for	Why include it
Logistic / Linear Regression	Baseline, interpretability	Fast, explainable baseline to beat
Random Forest	Non-linear patterns, tabular data	Strong out-of-the-box performance, handles messy features
XGBoost / Gradient Boosting	Competition-grade accuracy	Usually the best performer on structured/tabular data
(Optional 4th) SVM or Neural Network	Complex boundaries	Good comparison point if the top 3 tie closely

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Random Forest": RandomForestClassifier(n_estimators=200, random_state=42),
    "XGBoost": XGBClassifier(eval_metric="logloss", random_state=42)
}

results = {}
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    preds = model.predict(X_test_scaled)
    results[name] = {
        "accuracy": accuracy_score(y_test, preds),
        "f1": f1_score(y_test, preds, average="weighted")
    }

for name, metrics in results.items():
    print(name, metrics)
```

Step 8 — Compare & Justify Your Choice

Model	Accuracy	F1-Score	Training Time	Chosen?
Logistic Regression	0.81	0.79	Fast	—
Random Forest	0.87	0.86	Medium	—
XGBoost	0.90	0.89	Medium	✔ Best

> **MENTOR TIP**
> Never pick a model just because it has the highest accuracy on a small or imbalanced dataset. Check F1-score, precision/recall, and confusion matrix too. State clearly in your report why your chosen model wins on the metrics that matter for your problem.

Step 9 — Save the Final Model

```
import joblib
best_model = models["XGBoost"]
joblib.dump(best_model, "model.pkl")
joblib.dump(scaler, "scaler.pkl")
```

Step 10 — Build the Backend API

The backend loads your saved model and exposes a prediction endpoint. Build with either FastAPI (recommended) or Flask.

OPTION A (RECOMMENDED) — FASTAPI

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
import numpy as np

app = FastAPI(title="ML Prediction API")
model = joblib.load("model.pkl")
scaler = joblib.load("scaler.pkl")

class InputData(BaseModel):
    feature1: float
    feature2: float
    feature3: float

@app.get("/")
def home():
    return {"message": "ML API is running"}

@app.post("/predict")
def predict(data: InputData):
    values = np.array([[data.feature1, data.feature2, data.feature3]])
    scaled = scaler.transform(values)
    prediction = model.predict(scaled)
    return {"prediction": int(prediction[0])}

# Run with: uvicorn main:app --reload --port 8000
```

OPTION B — FLASK

```
from flask import Flask, request, jsonify
import joblib
import numpy as np

app = Flask(__name__)
model = joblib.load("model.pkl")
scaler = joblib.load("scaler.pkl")

@app.route("/")
def home():
    return {"message": "ML API is running"}

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    values = np.array([[data["feature1"], data["feature2"], data["feature3"]]])
    scaled = scaler.transform(values)
    prediction = model.predict(scaled)
    return jsonify({"prediction": int(prediction[0])})

if __name__ == "__main__":
    app.run(debug=True, port=8000)
```

Criteria	FastAPI	Flask
Auto docs (Swagger UI)	✔ built-in at /docs	Needs extra library
Data validation	✔ via Pydantic	Manual
Speed / async support	✔ Native async	Limited
Learning curve	Slightly steeper	Very beginner-friendly
Best for this project	✔ Recommended	Acceptable alternative

Step 11 — Build the Frontend

Choose Streamlit (fastest for ML students) or a JavaScript frontend (better if you want to show web-dev skills too).

OPTION A (RECOMMENDED) — STREAMLIT

```
import streamlit as st
import requests

st.set_page_config(page_title="ML Prediction App")
st.title("ML Prediction App")
st.write("Enter values below to get a prediction from our trained model.")

feature1 = st.number_input("Feature 1")
feature2 = st.number_input("Feature 2")
feature3 = st.number_input("Feature 3")

if st.button("Predict"):
    payload = {"feature1": feature1, "feature2": feature2, "feature3": feature3}
    response = requests.post("http://localhost:8000/predict", json=payload)
    result = response.json()
    st.success(f"Prediction: {result['prediction']}")

# Run with: streamlit run app.py
```

> **MENTOR TIP**
> Streamlit gets you deployed in hours, not days — ideal if your specialty is ML, not front-end engineering. Only choose the JavaScript route if you're comfortable with HTML/CSS/JS or want a more custom, recruiter-impressive UI.

Step 12 — Deploy to the Cloud

A project isn't finished until a stranger can open a link and use it. Pick a free-tier host:

Platform	Best for	Notes
Render.com	FastAPI / Flask backend	Free web service, auto-deploy from GitHub
Railway.app	FastAPI / Flask backend	Simple, fast deploys
Streamlit Community Cloud	Streamlit frontend	Free, one-click deploy from GitHub repo
Vercel / Netlify	JavaScript frontend	Great for static HTML/JS/React apps
Hugging Face Spaces	Full ML app (Streamlit/Gradio)	Popular with ML/AI portfolios
Docker + any cloud VM	Full control deployment	Use for advanced/bonus points

Step 13 — Connect Frontend to Deployed Backend

Once your API is live (e.g. https://your-api.onrender.com), update your frontend's request URL from localhost:8000 to the live URL, then redeploy the frontend.

> **FINAL DELIVERABLE FOR TRACK A**
> One live Streamlit/JS frontend URL + one live FastAPI/Flask backend URL (or combined into one deployed app), a public GitHub repository, and a 1-page project summary.

Step 14 — Documentation & GitHub

- Create a public GitHub repository with a clear folder structure: /data, /notebooks, /backend, /frontend.
- Write a professional README.md: problem statement, dataset source, models compared, chosen model + why, how to run locally, live demo link, screenshots.
- Add a requirements.txt for reproducibility.
- Include your name, cohort, and mentor credit (GworldSoft Solutions Limited / 3MTT) in the README.

4. Track B — Data Analyst: Power BI Dashboard

Data Analyst students build and publish an interactive Power BI dashboard, ending with a shareable URL rather than a deployed app.

Step 1 — Pick a Business Question & Dataset

- Choose a real, relevant dataset (sales, HR, finance, healthcare, Nigerian economic data, etc.) from Kaggle, NBS, or company/sample data.
- Define 3–5 business questions your dashboard must answer (e.g. "Which region drives the most revenue?").

Step 2 — Import & Clean Data with Power Query

- Get Data → import CSV/Excel/SQL source into Power BI Desktop.
- Open Power Query Editor: remove duplicates, fix data types, rename columns, handle nulls.
- Apply transformations: split columns, merge queries, unpivot where needed.

Step 3 — Build the Data Model

- Design a star schema: fact table (transactions) + dimension tables (date, product, region, customer).
- Create relationships in the Model view (one-to-many, correct cardinality).
- Build a proper Date table using DAX if none exists.

```
DateTable = CALENDAR(DATE(2023,1,1), DATE(2026,12,31))
```

Step 4 — Write DAX Measures

```
Total Revenue = SUM(Sales[Amount])

YoY Growth % =
DIVIDE(
    [Total Revenue] - CALCULATE([Total Revenue], SAMEPERIODLASTYEAR('DateTable'[Date])),
    CALCULATE([Total Revenue], SAMEPERIODLASTYEAR('DateTable'[Date]))
)

Top Region =
CALCULATE(
    MAX(Sales[Region]),
    TOPN(1, VALUES(Sales[Region]), [Total Revenue], DESC)
)
```

Step 5 — Design the Dashboard

- Use a clean layout: KPI cards at the top, trend chart, breakdown chart, and a detail table.
- Apply consistent brand colors, a clear title, and slicers for filtering (date, region, category).
- Add tooltips and drill-through pages for deeper analysis.

Step 6 — Publish & Share

1. In Power BI Desktop: Home → Publish → Publish to Power BI (sign in with a free/work account).
2. Open the report in the Power BI Service (app.powerbi.com).
3. Click File → Embed report → Publish to web (public) for a public shareable URL, or use Share to generate a link for organizational access.
4. Copy the generated URL and test it in an incognito window before submitting.

FINAL DELIVERABLE FOR TRACK B
One published Power BI dashboard URL, the .pbix file, and a 1-page summary of insights and business recommendations drawn from the data.

MENTOR TIP
"Publish to Web" makes your dashboard public to anyone with the link — do not use it with sensitive or confidential data. Use organizational sharing instead for private datasets.

5. Deliverables Checklist & Grading Rubric

Track A Checklist (ML / DS / AI)

- ✓ Dataset sourced and documented (Kaggle/UCI/etc.) with link
- ✓ Data cleaning + EDA notebook with insights
- ✓ 3 models trained and fairly compared with metrics table
- ✓ Best model selected and justified
- ✓ Model saved (.pkl) and loaded correctly in backend
- ✓ Backend API built (FastAPI or Flask) with working /predict endpoint
- ✓ Frontend built (Streamlit or JS) connected to the API
- ✓ App deployed with a live, working public URL
- ✓ GitHub repo with clear README and requirements.txt

Track B Checklist (Data Analyst)

- ✓ Dataset sourced and business questions defined
- ✓ Data cleaned in Power Query
- ✓ Data model built with correct relationships
- ✓ At least 3–5 meaningful DAX measures
- ✓ Dashboard designed with KPIs, charts, and slicers
- ✓ Dashboard published with a working shareable URL
- ✓ 1-page insights and recommendations summary submitted

Grading Rubric (100 points)

Criteria	Points
Data quality & problem definition	15
Analysis / EDA / model comparison depth	20
Technical build (API / dashboard correctness)	25
Deployment (live, working URL)	20
Documentation & presentation	10
Demo / defense & Q&A	10

6. Recommended Tools & Resource Links

Category	Tool / Resource	Link
Datasets	Kaggle	www.kaggle.com/datasets
Datasets	UCI ML Repository	archive.ics.uci.edu
Datasets	Nigeria Open Data	nigeria.opendataforafrica.org
Modeling	scikit-learn Documentation	scikit-learn.org
Modeling	XGBoost Documentation	xgboost.readthedocs.io
Backend	FastAPI Documentation	fastapi.tiangolo.com
Backend	Flask Documentation	flask.palletsprojects.com
Frontend	Streamlit Documentation	docs.streamlit.io
Deployment	Render	render.com
Deployment	Streamlit Community Cloud	streamlit.io/cloud
Deployment	Hugging Face Spaces	huggingface.co/spaces
BI / Analytics	Power BI Desktop (free download)	powerbi.microsoft.com
BI / Analytics	Power BI Service	app.powerbi.com
Version Control	GitHub	github.com

MENTOR TIP
Bookmark this page. These are the exact tools used daily by working Data Scientists, ML Engineers, and Data Analysts in Nigeria and globally in 2026.

7. About Your Instructor & Mentor

Godspower Uyanga

Lead AI & ML, GworldSoft Solutions Limited | Lead Data Scientist & AI Engineer, Interswitch Group | Mentor, 3MTT Nigeria

MSc Software Engineering · MSc Financial Engineering · MSc Artificial Intelligence Engineering

Godspower Uyanga leads AI and Machine Learning at **GworldSoft Solutions Limited** and serves as Lead Data Scientist & AI Engineer at **Interswitch Group**, in addition to mentoring students on the technical track of this capstone program in partnership with the **3 Million Technical Talent (3MTT) Nigeria** initiative. With a multidisciplinary academic background spanning software engineering, financial engineering, and artificial intelligence engineering, and hands-on experience delivering production AI/ML systems in the fintech industry, this program is designed to bridge the gap between classroom learning and the real, deployable skills employers expect from Data Scientists, ML/AI Engineers, and Data Analysts today.

Connect on LinkedIn: linkedin.com/in/godspower-uyanga-76aab01b0

A NOTE TO STUDENTS

Complete both the technical build and the storytelling around it — a great model with no clear README, and a great dashboard with no clear insight, are both incomplete projects. Ship something you're proud to put your name on.